Influence Maximization Analysis. Progress Report

Antoni Zyla Supervised by Nathan Griffiths

Contents

1 Introduction **3**
1.1 What am I studying? . . . 3
1.2 What do I hope to achieve? . . . 3
1.3 Why is this a good project? . . . 3

2 Background Research **3**
2.1 Models of Influence Propogation . . . 3
2.2 Working with large networks . . . 4
2.2.1 Naive Approaches . . . 4
2.2.2 Smarter Sampling Methods . . . 5
2.2.3 Methods for finding seeds . . . 5

3 Current Status **6**
3.1 Working with Networks . . . 6
3.2 Synthetic Networks . . . 7
3.2.1 Navigable Small World Graphs . . . 7
3.2.2 Power Law Degree Distrubted Graphs . . . 7
3.3 Cascade and Threshold Models . . . 8
3.3.1 Heuristics . . . 9
3.4 Evaluation of Sampling Methods . . . 9
3.5 Minimum Node Analysis . . . 12
3.6 Rate of Change Analysis . . . 13
3.7 Packaging . . . 14
3.8 Parallelisation and Performance . . . 14
3.9 Documentation . . . 15
3.10 Technical Content . . . 15
3.10.1 Code Structure . . . 15
3.11 Performance . . . 15
3.12 Data Visualisation . . . 16

4 Upcoming Goals **18**

5 Reflection **18**

6 Project Management **18**

7 Legal, Social, Ethical and Professional Considerations **19**

A Abstract **20**

B Problem Discussion **20**
B.1 Models 20
B.2 Sampling Methods 20
B.3 Heuristics 21
B.4 Application to research 22

C Objectives **22**

D Methods and Methodology **23**

E Timeline **23**

F Resources and Risks **23**
F.1 Resources 23
F.2 Risks 24

G Legal, Social, Ethical and Professional Considerations **24**

1 Introduction

1.1 What am I studying?

Influence is the idea of a concept or idea spreading throughout a network. Influence can be used as a proxy for the spread of infectious diseases, behaviours [1] or even for helping with monitoring infrastructure [15].

Finding a subset S of nodes from a network which have the greatest influence or spread is a well studied problem and there exist many models, algorithms and heuristics which help to efficiently achieve this goal. This is an active area of research with new papers being regularly published and studied.

1.2 What do I hope to achieve?

To find a heuristic which works well on a broad range of networks, performing better than others available for the same computing power or performing the same with less computing power required.

1.3 Why is this a good project?

Systematic methods such as taking a greedy approach to find some set of influential nodes may seem like a good way to achieve a 'good enough' result. Even a simple approximation such as greedy can take over 10 times longer than even an expensive heuristic approach whilst at the same time often producing a result less than or equal to the much much faster heuristic method.

The performance of any method for determining the seeds is highly dependent on the network topology, size, and required k. Being able to find a correlation between a topological property and an efficient seed selection method to produce high influences could for example allow for marketing campaigns to be made more efficient, more effective or cheaper to run. Study in this field is incredibly useful in working on social platforms, recommendation systems, trend analysis or network anaylsis to find emerging trends [10].

2 Background Research

I have considered a wide range of sources so far including research papers, doctoral theses, open source datasets, articles and open source code. Each piece of information is cited or linked as a footnote where appropriate.

2.1 Models of Influence Propogation

There are three main models through which influence is typically spread across networks.

1. Independent Cascade Model (ICC) - Each node can take a state out of Active, Inactive and uninfected. Each active node has a probability p to infect each of it's uninfected neighbors. Once a node has been infected this process starts again to all of it's neighbors, the node is in the active state once it is infected [10]. Once a node has attempted to infect all of it's neighbors it becomes an inactive node. This is the simplest of the models but also mimics real life well, weather it's the spread of a new software technology or a new restaurant becoming more popular.

2. Decreasing Cascade Model (DCC) - A modification of the ICC model where every attempt of a node to be influenced decreases the probability of it being influenced or infected [11]. This could be analogous to a common experience, overreaching of

advertisements or correspondence, such as marketing emails which have the opposite to intended effect due to annoyance.

3. Linear Threshold Model (LTM) - Every node v is given a probability of being turned p which is uniformly distributed across the interval $[0, 1]$. When the proportion of node v's neighbors which are infected is greater than or equal to p, v itself becomes infected [10]. This models well another common experience, a group reaching a common view in regards to something, whether it's the adoption of a new game, or even a software tool that becomes the 'industry standard' this principle is at play.

Various papers have described modifications of each of these models in a multitude of ways, multiple concepts can be spread wherein they can inhibit the spread of eachother by allowing a node to only adopt a single concept [2, 8]. Vaccination or heard immunity can be modelled in this manner [2].

There are other models which work in different manners, borrowing from biology models such as SIR for modelling diseases more closely [13]. These are somewhat able to be modelled using the cascade models however full parity is not possible for example going from infected back to uninfected is not a behavior that is considered with the cascades and would possibly have large effects on the effectiveness of certain seed selection strategies.

There are models where paths between nodes can be added or removed as part of the process [16], this can for example be used to model the evolution of social networks, rather than studying networks which are at a fixed point in their evolution. Influence can be studied over these evolutions however this is an additional very large jump in complexity and scope which would be out of scope for this project.

2.2 Working with large networks

Some of the networks I will be studying are made up of tens of thousands of nodes with hundreds of thousands of edges, working on networks of this size is very resource intensive.

Sampling networks is a way of preserving the topological features and allows us to create strategies which generalise well to networks of a much greater size.

2.2.1 Naive Approaches

A simple natural approach could be to take a node at random from the graph, call it v, and then perform a breadth first search in layers across the graph. When a high degree node is encountered then each of it's neighbours would also be added, this will inflate the average degree of the network by tending to sample a subset with a higher mean degree [17].

Working outwards from a start node also does not preserve the distribution of shortest path distances between nodes or clustering coefficients well. For node located close to the root these values will remain quite accurate but as we move further and further out a node is less likely to appear before we have no more nodes needed leading to a lower shorter mean path length and a higher typical clustering coefficient over other methods [6].

Some papers describe BFS as adding the entire layer from which it explores, leading to oversized samples [5]. Taking BFS sample of 1000 nodes in the Enron dataset it produces on average a sample of over 12 thousand nodes which is in no way comparable to another method's 1000 node sample. This is not useful when comparing like for like network sizes and their respective topological features such as diameter which are closely dependent on overall network size.

Snowball sampling is a very simple variation on breadth first search, once the addition of the final layer would take the total number of sampled nodes over the threshold, nodes are simply added one at a time until the exact number of nodes is reached [5].

A random walk is another simple strategy for exploring a graph. Working on the basis of a standard random walk, starting from a node v there is a $\frac{1}{deg(v)}$ probability of choosing

the next node to be u where u is one of the neighbours of v. intuitively RW will suffer from the same problems as BFS with finding higher degree nodes much more often than lower degrees [6], as a higher degree node has more connections that lead to it.

2.2.2 Smarter Sampling Methods

Utilising modified random walks is a standard method of achieving accurate and unbiased samples of large networks such as facebook [6]. Metropolis Hastings Random Walks and Random Walks with Delayed Acceptance (MHRW and MHDA) are two common methods used [6, 7]

2.2.3 Methods for finding seeds

There are a few methods which don't rely on a heuristic or rule of thumb to determine the seed nodes.

1. Brute Force - $\binom{n}{k}$ possible seed sets from which to choose from. This is incredibly inefficient and on the order of $O(n!)$ cascades.
2. Greedy - Run a cascade from each node and then choose the k highest influence nodes [11] giving a $1 - 1/e$ approximation of opt whilst being $O(n)$ cascades.
3. Cost Effective Lazy Forward (CELF) - A much faster greedy based algorithm working on only recomputing data for each node when needed [15].

The above approaches are however incredibly slow to compute for any large networks. Utilising a heuristic or approximation between a characteristic of a node and it's significance in spreading across the network can be a much more performant way of extracting similar results but at a computationally feasibly scale.

As detailed in section B.3 of the specification there are a large amount of heursitics which work well as a proxy for determining how influential a node is as shown by figure 5.

I have primarily found two main groups of heuristics, simple heuristics which consider a node and potentially it's neighbours and some more complex heuristics which consider how a node fits into the wider structure of a network.
As is quite easily seen in figure 5 and 6 these two main types are quite apparent, the rates of growth and behaviour at low seed sizes is consistent between the propogation models and network sizes with the grouping behaviour even more visible in figure 4.

3 Current Status

Overall good progress has been made, most of the desired objectives have been achieved. The codebase is in good shape, code could be written more clearly, cleanly and modularly however at this stage it works well and effectively. Documentation from docstrings is automatically generated into a navigable website that links between functions classes with examples of how use each function.

Objective	Status	Notes
1. Import and Export of Graphs	Complete	Not implemented unit testing
2. Generation of Synthetic Graphs	Complete	
3. Influence Models	Complete	
4. Sampling Methods	Partially Complete	TODO: Albatross Sampling
5a. Cost based analysis heuristics	Incomplete	Would be the most complex, may not implement if not needed
5b. Minimum Nodes heuristics	Complete	
5c. Rate vs Absolute change heuristics	Complete	
6. Packaging	Complete	
7. Parallelisation	Complete	
8. Documentation	Partially Complete	

Table 1: Current Status Summarised

3.1 Working with Networks

As listed in objective 1 my project needs the ability to import and export networks. I have implemented the ability to import networks from a file as commonly found online [], custom delimiters are able to be used. To make getting started with my program quick and easy I have included some networks inside of the python package, they are listed below in 2 along with some information about them.

Name	Nodes	Edges	Mean Deg.	Cluster. Coef.	Diam.	Centralization
Enron	36692	183831	10.020	0.497	13	0.00725
P2P	10876	39994	7.355	0.006	10	0.001899
Airports	1574	17215	21.874	0.504	8	0.004334
Twitter	81306	1342310	33.019	0.565	7	0.002114

Table 2: Metrics for included networks

1. **Enron** Email network from Enron an energy company that shut down in 2001 due to fraud. Each email represents an edge, from the edgeset the network is then constructed, created by SNAP [1].

2. **P2P** Gnutella is a peer to peer (p2p) file sharing protocol created in 2000, this network compiled in 2002 relates each host as a node and any connection between two hosts as an edge. The protocol created some interesting topologies, with an overall goal of keeping the longest shortest path between any two nodes of at most 4 it relied on a

[1] https://snap.stanford.edu/data/email-Enron.html

few nodes to act as an intermediate layer (Ultrapeers) between the majority of clients [2].

3. **Airports** A dataset of all flight traffic between american airports in 2010 [3]. This dataset is useful as it is a large network with likely a hub and spoke topology where certain nodes have much much higher traffic or degrees than others. It is visualised in figure 3 and shows this quite well.

4. **Twitter** A large collection of circles, a feature which allowed users to specify to whom their tweets would go to, restricting these tweets from being public. It was obtained by crawling the website in 2012 by the Stanford SNAP research group [4]

3.2 Synthetic Networks

Many papers commonly compare synthetic networks to sampled networks from realworld structures, it is important for me to be able to replicate these trials.

3.2.1 Navigable Small World Graphs

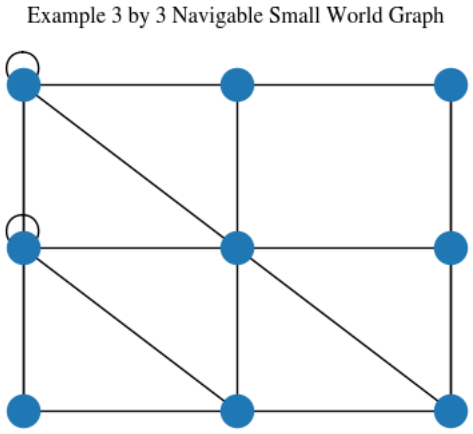


Figure 1: Example of a 3 by 3 Navigable Small World Network

A small world network is one that has a low mean shortest path between any two nodes and a high clustering coefficient [12]. This means that there is a high likelihood that any two nodes in the network are able to form a path between themselves using very few nodes, an example of this is the six degrees of separation theory.

There are a few parameters [5] that allow for changing the shape and properties of these graphs, when applicable to compare to given results I will use the same or closest values.

3.2.2 Power Law Degree Distrubted Graphs

A power law graph means that the distribution of node degrees follows a power law distribution [4], a very high degree node is much much rarer than a typical degree node following as the name suggests a powerlaw. This is also referred to as scale free and is a very close

[2] https://rfc-gnutella.sourceforge.net/src/rfc-0_6-draft.html
[3] https://www.transtats.bts.gov/Homepage.asp
[4] https://snap.stanford.edu/data/ego-Twitter.html
[5] https://networkx.org/documentation/stable/reference/generated/networkx.generators.geometric.navigable_small_world_graph.h

model of the airports network, academic paper authorship or other social networks. To form a graph with k nodes a sequence of k elements is created by sampling a power law distribution, the ability to form a graph from this sequence is then checked. The graph is then initialised from a randomly generated graph with node degrees that follow this sequence.

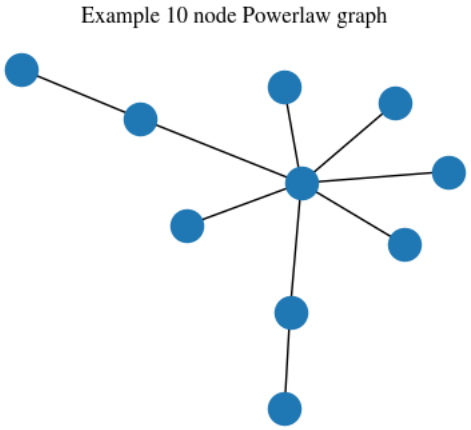


Figure 2: Example of a 10 node powerlaw graph

Comparing the above image (Figure 2) to the Airports in figure 3 (Smallest of the included graphs) graph visualised with just edges some similarities are easily visible.

3.3 Cascade and Threshold Models

As written in point 3, Independent Cascade, Decreasing Cascade and Linear Threshold models have been implemented. Code for automated running of trials, testing of heuristics on each of these has been written. The required computations perform well when ran on the department's compute cluster. Data is output in both tabular and graphical formats, below is an example of some early runs.

There are a few things we can garner from figure 5:

1. Degree based heuristics are very effective for how quick they are to compute.
2. Clustering Coefficient is very effective, however only after a certain number of seeds, this could be due to how clustering and degree are presented, a high degree node may be connected to a significant proportion of the network whereas a node with a high clustering coefficient doesn't necessarily have a high possible number of effected neighbors.
3. The asymptotic behavior of degree based heuristics is likely showing that the degree distribution of the network is one with a small number of very high degree nodes tightly packed. Requiring a large number of new seeds to make significant strides in terms of spread may show that high degree nodes are closely clustered together.
4. There being minimal spread beyond the based starting nodes in the decreasing cascade and a convergence of the heuristics demonstrates that there may need to be some adjustment to the loss in turning a node per attempt.
5. The linear threshold model shows a similar pattern to the independent cascade model with regards to there being two distinct groups of heuristics in terms of their results and behaviour at low seed sizes.

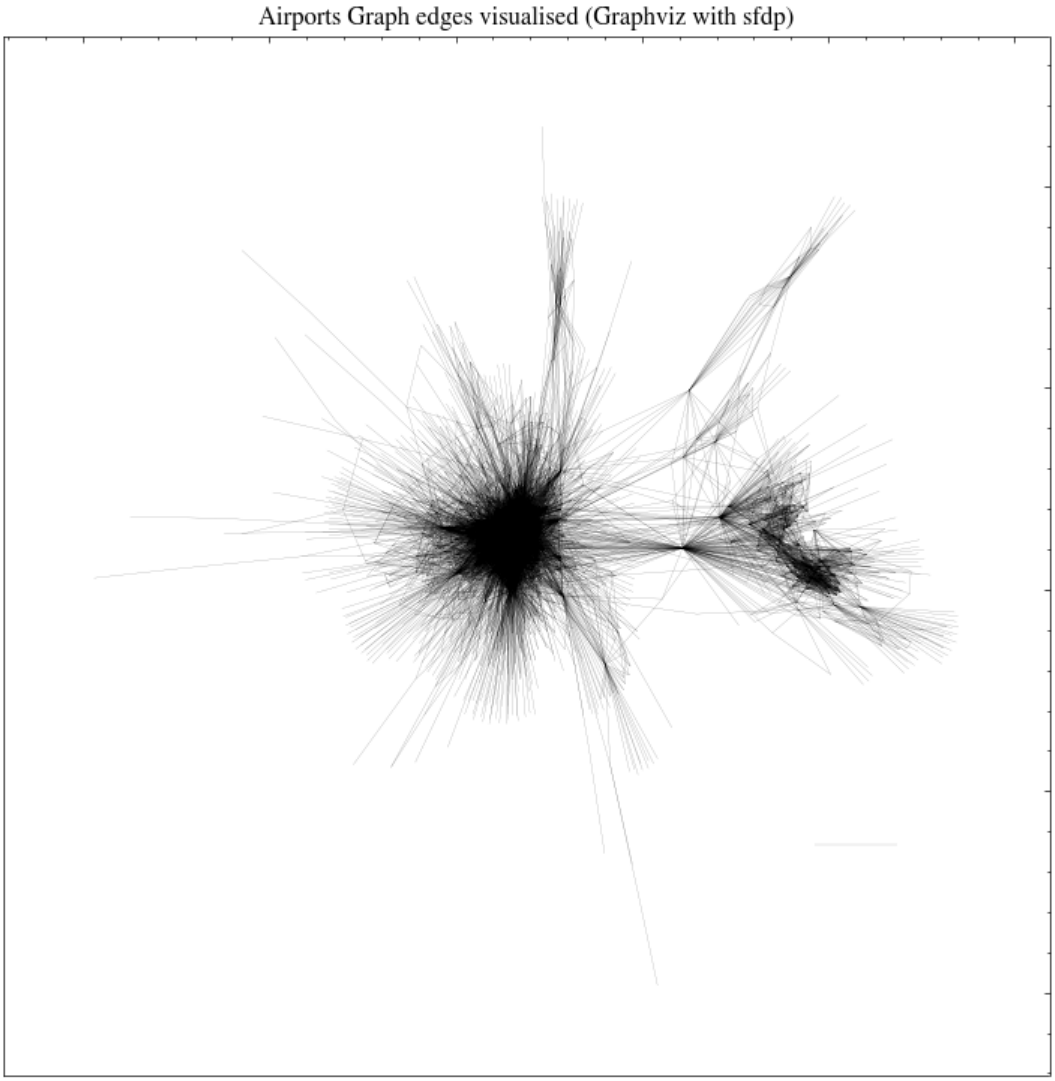


Figure 3: Airports Graph Edges Visualised

Following figure 5 I implemented a few more heuristics as well as the naive greedy algorithm and the improved CELF version.

3.3.1 Heuristics

I have most of the described heuristics from part B.3 of the specification however I have not implemented numbers 7, 8 and 10 or Mean Shortest path length, Edge Embeddedness and Overlap. I am in the process of implementing these and using them to compare to others. Some heuristics are very complex to calculate empirically and there exist approximations which I am also looking into particularly for Betweenness and Eigenvector centrality.

3.4 Evaluation of Sampling Methods

As written in section B.2 of the specification, sampling methods 1,2,4 and 5 have been implemented. Tables similar to 2 and 3 are quickly output with data about their mean and

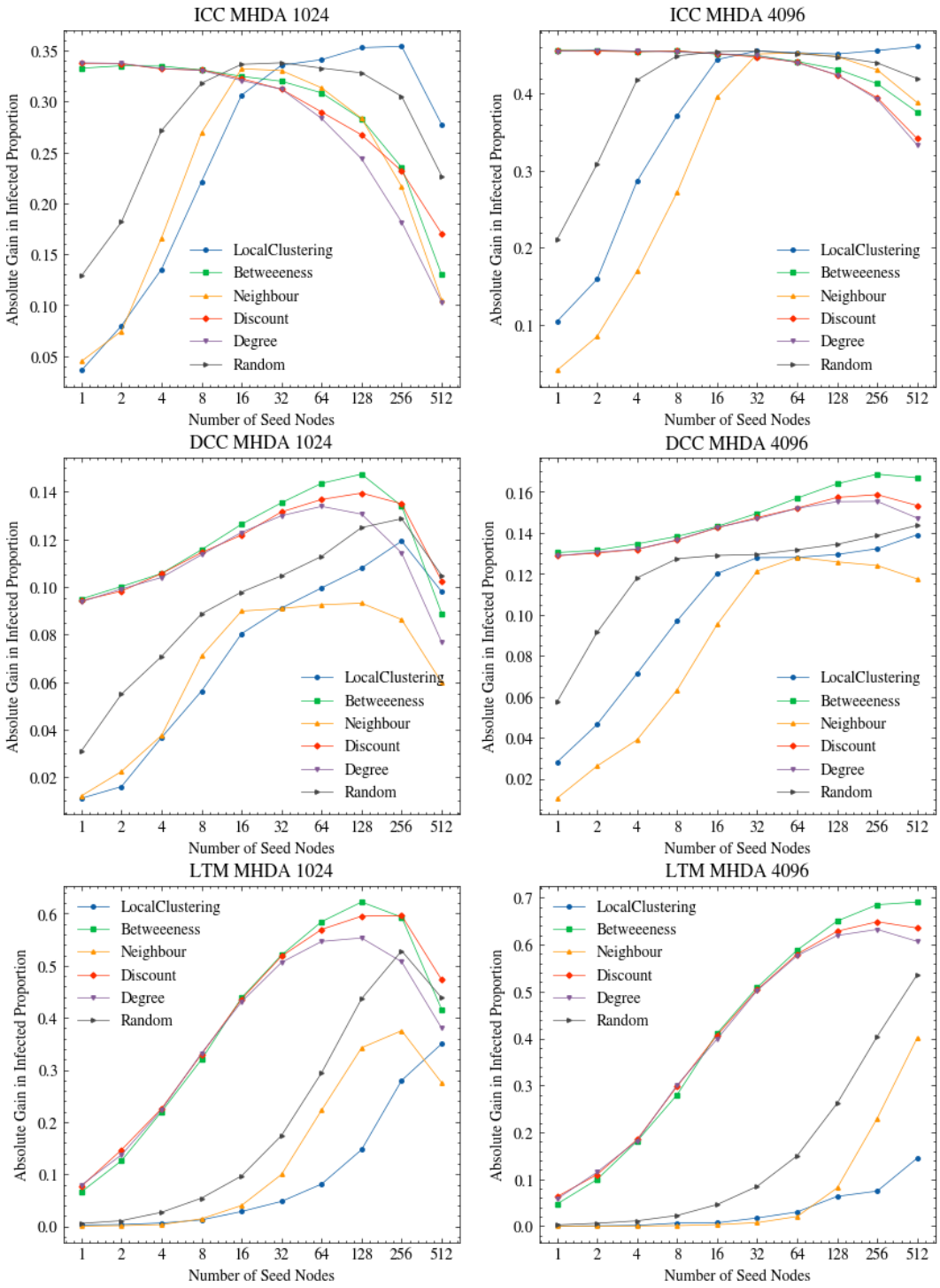


Figure 4: Absolute Gains in Influence Compared

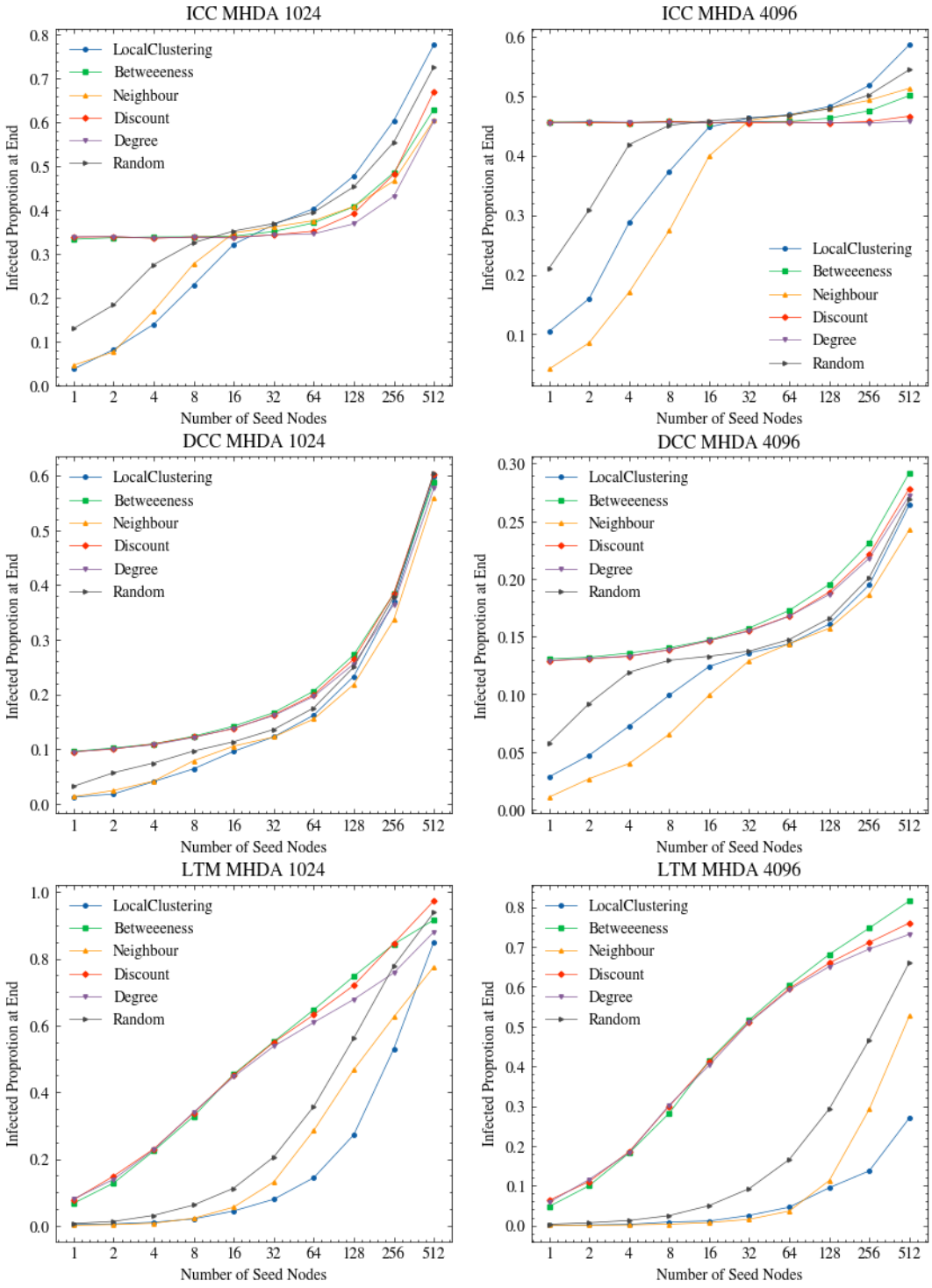


Figure 5: Early Running of Models with heuristics

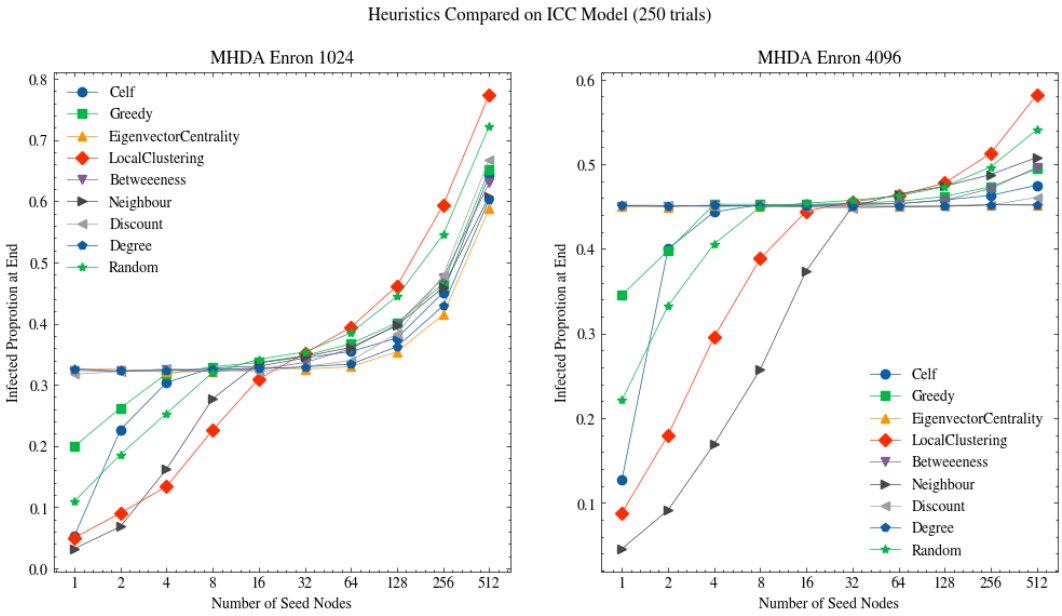


Figure 6: Currently Implemented Heurstics on ICC

standard deviations on any dataset.

Method	Nodes	Edges	Mean Deg.	Cluster. Coef.	Centralization	Diam.
Airports	1574	17215	21.9	0.504	0.004	8
MHDA	1000	13586 (301)	27.2 (0.6)	0.50 (0.01)	-0.01 (0.001)	8 (0.3)
MHRW	1000	13576 (250)	27.2 (0.5)	0.50 (0.01)	-0.01 (0.001)	8 (0)
SNS	1000	15216(313)	30.4 (0.6)	0.60 (0.02)	-0.01 (0.001)	4 (0.5)
BFS	1281 (132)	16094 (652)	25.3 (1.7)	0.55 (0.02)	-0.001 (0.003)	5 (0.7)

Table 3: Various Sampling Methods on the Airports Dataset (15 Runs)

As shown in the above table 3; MHDA clearly shows that it is the closest method in terms of preserving the topology of the networks.

These results are also consistent with the data found in [5–7].

In the project specification I outlined inclusion of Random Walk (RW) and Albatross (AS) sampling, they are not included in the analysis here as I have not yet got a fully working and complete implementation that I am happy with.

3.5 Minimum Node Analysis

Working out the minimum number of nodes for a given target proportion is simple if using a naive approach. For a given network $G = (V, E)$ where $|V| = n$ and $|E| = m$ it would take $O(n)$ cascades to find such seed set size.

There is however a smarter and more efficient way of doing this computation. Taking an ordered expnonetially increasing sequence of $S_n = (2^i, n)$ for $i \in \{0, 1, .., \lfloor \log_2(n) \rfloor\}$ to denote the seed sizes we want to look at. Noting that the sequeunce S is of length at most $\log_2(n) + 1$. Starting by iterating performing cascades from S_1 until we find S_k where the influence spread of S_k is greater than or equal to the required value. As the influence is

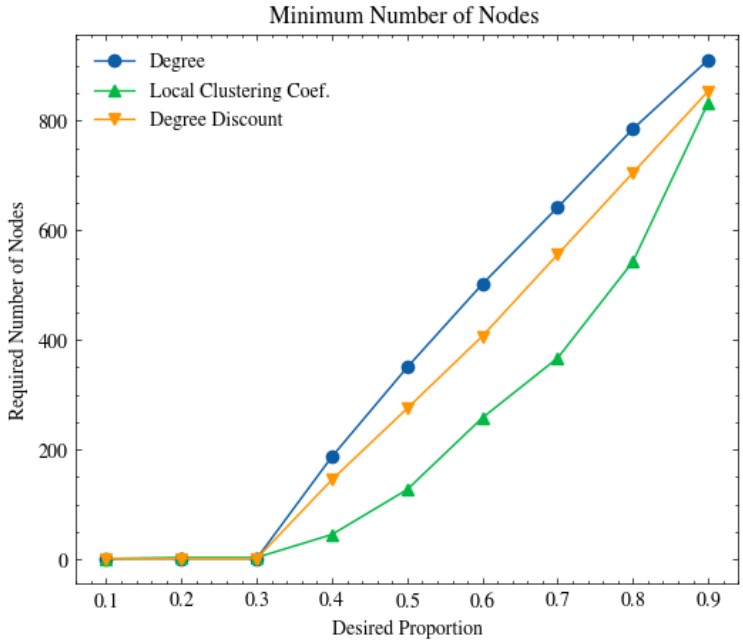


Figure 7: Minimum number of nodes analyisis, Enron MHDA Sample 1024 Nodes

proprtional to a given seed set size and also increasing this implies that the influence of S_{k-1} must be less than the required value. Creating another ordered increasing sequence $A = (S_{k-1}, S_{k-1}+1, ..., S_k)$. Noting that the sequence A is of length at most $\frac{n}{2}$. Performing binary search to the find the first element that is greater than or equal to the required value required taking $O(\log_2(\frac{n}{2}))$ cascades. Combining both the large step and the small steps results in $O(\log(n))$ cascades being performed. It may be easier to run binary search over the whole range $(0, 1, ..., n)$, the reason for going with the alternate approach described above is in the case of a low required target propogation.

On a large network the cascades are a costly computation, some networks with high degree values can achieve almost $\frac{1}{3}$ of their total nodes infected by a single seed node. Iterating upwards turns this into a very quick computation rather than performing $\log(n)$ costly cascades to just find the first element.

As shown in figure 7 the patterns between how different heuristics behave on similar datasets begin to become quite clear, with the degree based heuristics being very fast growing but then tailing off compared to local clustering where the growth is slower but steadier and overall requiring less nodes for the same value.

3.6 Rate of Change Analysis

Running each of the models returns a triplet of the number of iterations, the final influenced graph, and an array of influenced proportion values in the range of 0 to 1 for each iteration. This is then easily plotted as seen in figure 8.

As seen in figure 8 analysis around the rate of change of influence (the gradient) of different heuristics is simple whilst also comparing their absolute spread over time.

Attempting to find an optimal strategy if you only have 2 iterations is easily shown with this graph that a degree based approach would work much better than a clustering coefficient one even if the clustering coefficient would be much better if allowed to play out.

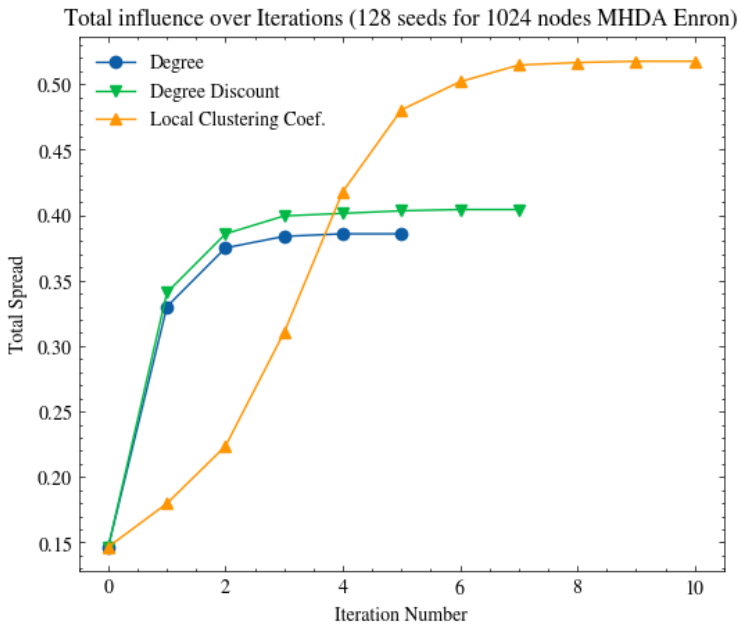


Figure 8: Influence by Iteration

3.7 Packaging

I am utilising an open source python library packaging tool called Hatchling [6], allowing me to package the entirety of the code into a `.tar.gz` and a `.whl` distributable. Python3.10 and above is required due to the introduction of union types in the type annotation system. I have attempted to use PyPy [7] however there were some issues with the multiprocessing code so I have decided to not focus on this aspect.

3.8 Parallelisation and Performance

Performing hundreds of cascade processes on tens of networks on tens of sample methods with tens of different seed sizes requires a large amount of computation. Each task such as run a cascade on this network with this sampling method with this many seeds using this heuristic is created as a tuple of inputs into an array, using the `Pool.starmap` [8] function it's easy to perform a catchall function on these and get the results using an arbitrary number of threads to complete the processing.

One of the main challenges with regards to running these processes quickly is sampling of large scale networks, sample methods such as MHDA require a large amount of computation steps for each node to actually expand the size of the current set. Saving a set of previously sampled networks and then running cascades on each of them is a simple way to make the overall performance of generating results more efficiently whilst also allowing for consistent results with fewer trials.

[6]Link: https://pypi.org/project/hatchling/
[7]Link: https://doc.pypy.org/en/latest/
[8]https://docs.python.org/3/library/multiprocessing.html

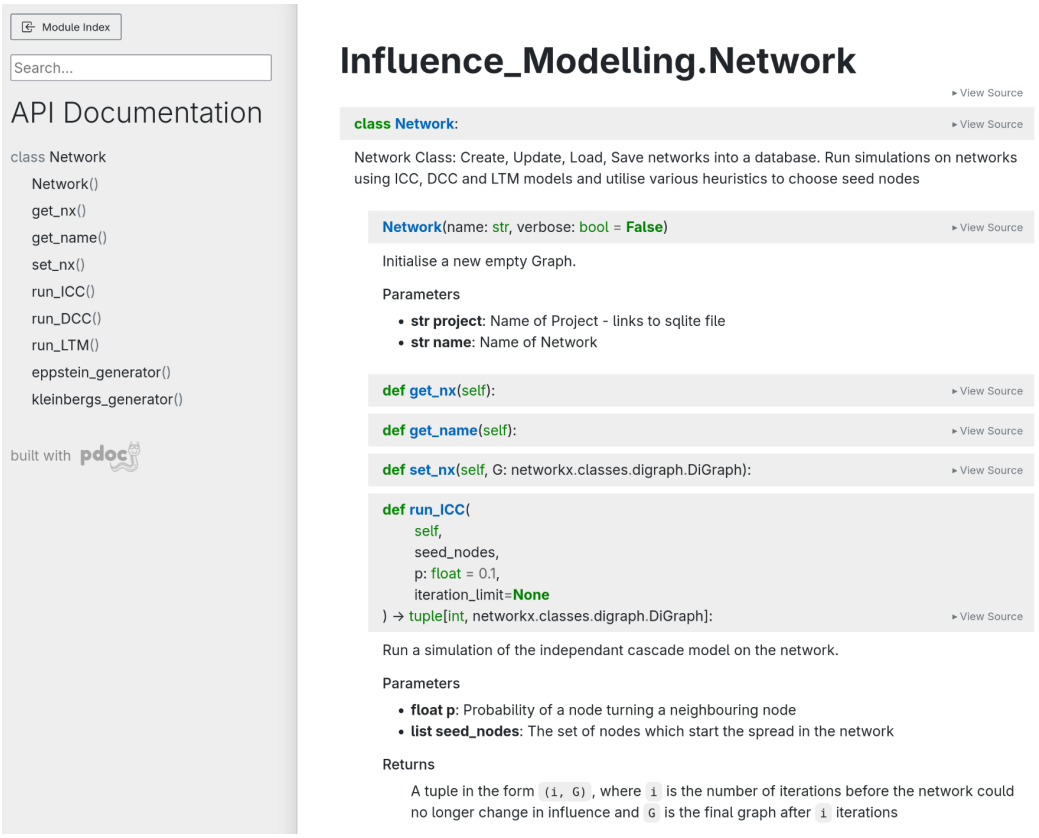


Figure 9: Example of automated documentation site

3.9 Documentation

A fully searchable and navigable set of static web pages is generated, in the future this could be embedded within a wider set of documentation including systems design, user guides, troubleshooting guides and various other resources.

3.10 Technical Content

3.10.1 Code Structure

The primary code is structured in an object oriented fashion, for example a network is a class which is then extended into a network that reads or writes csv files. Each set of functions is split into it's own file and can be imported separately.

My test scripts are standalone scripts that simply require the packaged library to be installed, these are the scripts which produce the results as displayed in all the tables and figures so far in this document.

3.11 Performance

For every run of a cascade, sampling method or any action I record the time taken, if running over a large number of samples I also automatically take the mean and standard deviation of the running time and this is an extra column at the end of each of the tabular outputs for any aggregate process.

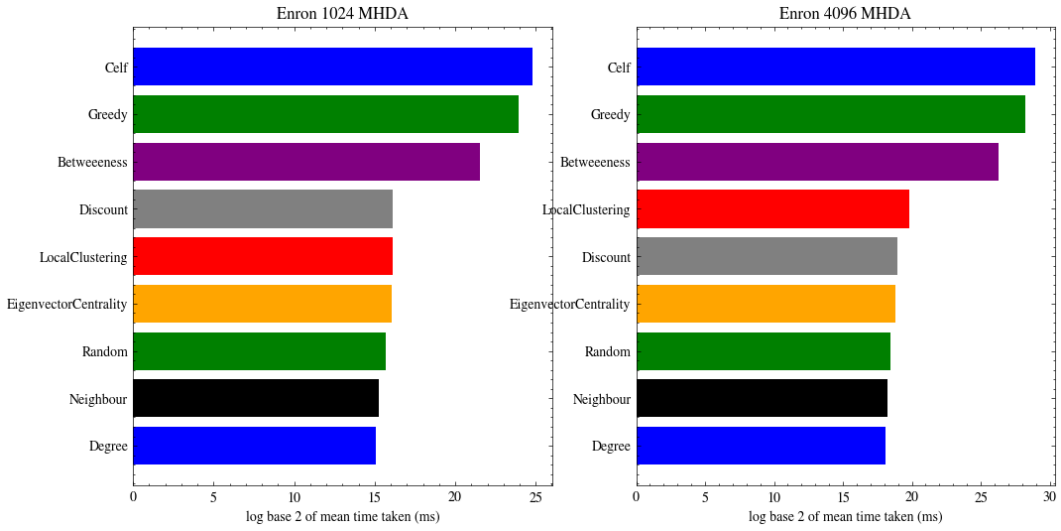


Figure 10: Mean Log Base 2 of time to compute each line in figure 6

Interestingly here CELF takes more time than Greedy to work on the same dataset. This is likely to be due to the overhead of using Python rather than a compiled language, array access in C or C++ is in effect offsetting a pointer to the array by the index multiplied by the size of the datatype. In python accessing a single element requires following the pyobject that makes up the arrays to get the value. This is much much slower when dealing with multiple array accesses especially two dimensional chained accesses as in the CELF implementation over just Greedy.

3.12 Data Visualisation

Something that is not immediately clear is how to best present data to easily and accessibly show the results and conclusions I am trying to make. I did not set out any clear requirements for this as at the time of writing the specification I did not yet know the best ways of presenting data and what ways would be most relevant to my results.

Showing a busy graph with lots of lines may be good for a vague overview or rough ordering of performance characteristics of each heuristic but it is not very useful for detailed close comparisons.

Following on advice from my supervisor I am in the process of implementing more intuitive visualisations. Those listed below are what I have currently implemented and how they help with conveying information.

1. Final spread for each seed size and heuristics - Allows for easily seeing how much a certain strategy will spread on average (Like in figure 5 or 6).

2. Absolute gain in influence for seed size and heuristics - Shows how much a strategy will gain by subtracting the starting proportion (Like in figure 4).

3. Tabular network data statistics - Useful for comparing sampling methods as well as seeing how they compare to the base network (Like in Table 3).

4. Offset spread graphs using one of the lines as new 0 line and giving the others as above or below to show relative orderings (Like in figure **??**).

5. Minimum Number of Nodes for a certain infected proportion - Figure 7

Some new ways of presenting data that I have been considering and am partway through implementing are:

1. Intersections of seed sets generated by different strategies.
2. Proportional gain visualisations for different strategies - This is something that I have implemented but it doesn't produce clear or easy to understand graphs. On small seeds that produce an incredibly high change in influence graphs like 4 this seem to fall apart when taking a log or dividing by the original seed proportion. Taking this into two axes with different scales could be a good way to display.
3. Histogram or similar diagram for the degree or connectivity of a set of nodes or network.

All graphs are generated using matplotlib [9] for the graph generation and scienceplots for ensuring consistent styling [10].

[9] https://matplotlib.org/
[10] https://github.com/garrettj403/SciencePlots

4 Upcoming Goals

i would like to explore an iterative approach based on a combination of greedy and heuristic methods. Choosing seeds iteratively by degree and not including those that the current set has spread to over x iterations could be a simple start to this.

Similar methods have been described in some papers [] and show promise so I would like to explore this direction.

I want to investigate further into the relationships between groups of nodes generated by different algorithms, for this I will need to program 1 for analysing this effectively. I would like to see if I can find any sort of pattern or correlation which would then inform my direction in attempting to find a better heuristic for some type of network.
I have explored some ideas regarding taking combinations of nodes that are both in a large cluster as well as those with a high betweenness, the results seem to be very network topology dependent creating a large variance in performance of the same algorithms with the difference stemming from individual network and sample variances. I would like to see if I can find out the exact topological structure that makes up this difference.

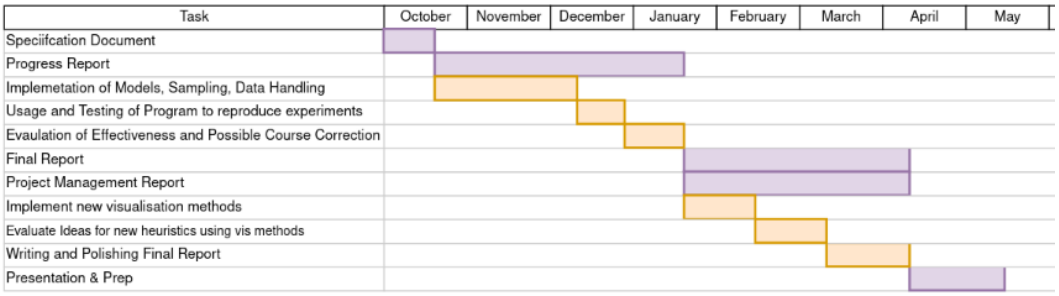


Figure 11: Updated Gantt Chart with new goals

5 Reflection

In terms of the technical goals the project is doing quite well. Most functions are complete and work as intended, results are able to be generated and presented in graph or tabular form quite quickly and efficiently.

I am concerned about my ability to actually create a new heuristic which will outperform those which have already been studied.

Primarily in and around the middle of the first term I started to focus on my graduate job search which not only took a lot of time but also made me quite demotivated in general. I find that when I have a new task to focus on I will hyper focus on that task and disregard my other obligations.

6 Project Management

The project was well suited to an incremental development strategy with fixed objectives at specific points in time with clear expectations.

All code versions, results and documentation are held in a git repository. Stored on GitHub with my personal computer and the DCS lab machines also holding up to date copies as I work on them.

To keep track of immediate tasks and goals I used a simple list in my notebook and by blocking time in my google calendar during the weeks.

7 Legal, Social, Ethical and Professional Considerations

As per section G of the specification there have been no changes to this.

Appendix: Project Specification

A Abstract

A network is a weakly connected directed graph about which we reason. Influence is the measure of how ideas or properties spread through this graph based on certain models; choosing the seed set of nodes from which to begin the model determines how quickly and how far the ideas will propagate. Social networks [18], infectious diseases and behaviors [1] can be studied by using such an approach and can be used to inform decision making in many areas.

Finding a subset of nodes of the network which produce the greatest influence spread after a given time period could be used to determine those who'd best promote a product or be at greatest risk of spreading an infectious disease [1]. Finding this subset is difficult and known to be NP-Hard, as a reduction from Vertex Cover [11]. Using certain heuristics such as the degree of a node to choose the seed nodes is significantly more efficient running in polynomial time [3].

Using real world networks to perform this analysis on is not only difficult to obtain but also very large and computationally intensive [17]. I will aim to create a tool which can be used to evaluate and compare different heuristics and sampling methodologies against different types of networks. I will then use this tool to find a more representative sampling method on networks, or a heuristic which works more effectively on a certain set of topologies.

B Problem Discussion

I will define a network upon which the models below will work on as a weakly connected directed graph. Doing so allows for modelling of networks in which the relationships are bidirectional (undirected) by simply creating two edges to represent that relationship, and for modelling directed networks such as email chains that have a clear sender and recipient. To be used for analysis, the network needs to be connected, as an idea cannot directly propagate between two disjoint components.

B.1 Models

1. Independent Cascade Model (ICM) - a node can be 'active' or 'inactive' and has a constant probability of infecting each of it's neighbours. Every infectious node has one attempt to infect it's neighbours before becoming inactive [11].

2. Decreasing Cascade Model (DCM) - an extension of the IC model the probability of infecting a node is determined by previous attempts, modelling how subsequent attempts are less likely to be successful after failed attempts [**?**].

3. Linear Threshold Model (LTM) - A model where each node has a certain 'threshold' required to infect it, considering the cumulative effect of the already infected nodes on the node determines if it will become infected [11].

B.2 Sampling Methods

Some real world datasets are far too large to efficiently compute these models on. Sampling these networks allows us to find a smaller network that encapsulates their features but is much more efficient to perform computations on. Those listed below I have researched and feel would be able to implement and use to evaluate against one another, as research progresses I may choose to add more methods or to not use a certain method.

1. Breadth First Search (BFS) - A natural exploration of the graph from a random node in layers [5].
2. Snowball Sample (SNS) - An extension of BFS which will stop once the desired sample size is reached [5].
3. Random Walk (RW) - Starting at a random node in the network chooses where to go onwards based on the the node's neighbours being chosen at random.
4. Metropolis Hastings Random Walk (MHRW) - A random walk with a modification to the probability function used to determine the subsequent node. This is to generate a uniform distribution of node degrees by favouring lower degree nodes [7].
5. Metropolis Hastings Random Walk with Delayed Acceptance (MHDA) - A further modification of MHRW in which the probability of backtracking to the previous node is reduced [14].
6. Albatross Sampling (AS) - A random walk with a chance of jumping to other locations in the network to find connected components [9].

B.3 Heuristics

Choosing which nodes to start the models on will affect how quickly the influence will propagate and how far it will spread to. A greedy approach may be close to optimal, however on larger datasets is inefficient and potentially intractable [2]. The following are some of the methods by which I will investigate, during the research phase I may add more or I may exclude some.

1. Random - As a baseline choose k random nodes.
2. Degree - Choose nodes with the highest degree.
3. Degree Discount - A modification of the degree heuristic but with a lower chance of choosing a node if it's neighbour is in the seed set [3].
4. Local Clustering Coefficient - The extent to which neighbours are connected to one another [5].
5. Average Neighbour Degree - Measuring the average degree of a node's neighbours may allow for communicating with high influence nodes [5].
6. Betweenness Centrality - Measuring how many shortests path's in the network pass throuhg a point [5].
7. Average Shortest Path Length - A measure of centrality by taking the mean length of the shortest path from the node to every other node [5].
8. Edge Embeddedness - For an edge (u, v) is the size of the intersection of the neighbourhoods of both u and v [5]. This serves as a measure of how tightly packed a network is around a certain area.
9. Edge Overlap - For an edge (u, v) the edge overlap is the ratio of it's embeddedness to the size of the union of the neighbourhoods of u and v. This can be used to determine the 'location' of an edge, for example if it connects connected components it may have a low overlap whilst a very high embeddedness may indicate not only is the agent well connected inside it's cluster it's also connected to other clusters [5].
10. Traditional Greedy Algorithm (Select Top k in [18]) - Choose greedily from nodes which have the highest influence [11]. This is inefficient due to requiring running a cascade for each node. It approaches within $(1 - 1/e)$ of the optimal solution so will be used to compare against heuristics.

B.4 Application to research

Utilising the above concepts, possible pieces of research in this area may include:

- Finding a more effective or representative heuristic for finding seed nodes in a network.
- Optimising a heuristic's running time whilst maintaining performance in regards to representation.
- Finding a more representative sampling method for varied network topologies.
- Finding a correlation between certain topological features and the effectiveness of one sampling method or heuristic over another, particularly in relation to that network feature.

C Objectives

The exact goal of my project is not fully decided, however there are some requirements for the code. With these features I would be able to perform much of the computation and comparison of various papers. Ideally I would be able to find something new or improve on existing techniques as described in section B.4.

1. Facilitate import and export of different graph formats.
2. Generation of synthetic graphs using graph generators for consistent analysis when real world datasets could be too large to run quick computations on.
3. Running and evaluation of the Independent Cascade Model, Decreasing Cascade Model and Linear Threshold Model. Allowing for evaluation of different heuristics against these models on different graphs.
4. Creation and evaluation of new sampling methods.
    - Evaluate the running time of sampling algorithms in practice, finding the real world constant factors.
    - Allow for evaluation of influence spread on the same network with different samples and how they affect it.
5. Creation and evaluation of new heuristics for selection of seed nodes against benchmark algorithms.
    - A 'cost' based strategy could be to give each algorithm a total cost, and each node has a value with the aim to find the subset that produces the greatest spread from nodes within a certain total 'cost'.
    - A 'minimum nodes' strategy to meet some spread metric, for example finding the lowest number of nodes that will produce a $\delta\%$ spread after x iterations.
    - Investigation of heuristics focused on overall growth in influence or rate of change in influence - this would involve charting the growth rate of various heuristics against eachother with trendlines etc.
6. Python **whl** and **tarball** packaging to allow for easy installation on various systems.
7. Programming with parallelisation in mind. Some results may take hours to compute on a personal device but are able to be ran in parallel on a cluster or more powerful system. I will attempt to use the batch compute system to produce any results.
8. Fully document all code, usage examples, parallelisation examples, and everything required to produce any results in my final report.

D Methods and Methodology

The project will be broken up into three main stages; research, implementation, and evaluation. During each stage I will be in contact with my supervisor to ensure that I am on the right track and to keep me accountable.

1. The research stage will require a detailed literature review of articles and papers pertaining to network influence modelling, methods, heuristics and sampling. I will identify the validity and applications of each source and find whether there are any gaps in the presented knowledge to research further. I will repeat this process until my supervisor and I are happy the full scope of the project has been thoroughly reviewed.
2. The implementation stage of the project will require a methodology that can be adapted to changing requirements or a shift in focus based on information learnt in the later stages of the research phase. I will use an agile-like methodology, of one to two week sprints determined by meetings with my supervisor, with any changes to requirements determined during these meetings with future sprints based on the previous.
3. I will use my framework to replicate the results of the researched papers, showcasing how to create new sampling methods and heuristics, and how to compare the results of different sampling, heuristics and models on different networks.
4. Finally, I will attempt to improve upon a sampling method or heuristic and use my framework to do so. This will demonstrate that my tool is applicable to real world research uses and potentially find something interesting or noteworthy.

E Timeline

The schedule is not fully decided at this point as the research phase is yet to be completed and the further points maybe change. Some points such as writing the report in the month leading to the deadline are certain, however the contents and direction of that report is not yet concrete. The purple bars on the gantt chart are the deliverables and the yellow bars are the main sub-tasks associated with those deliverables.

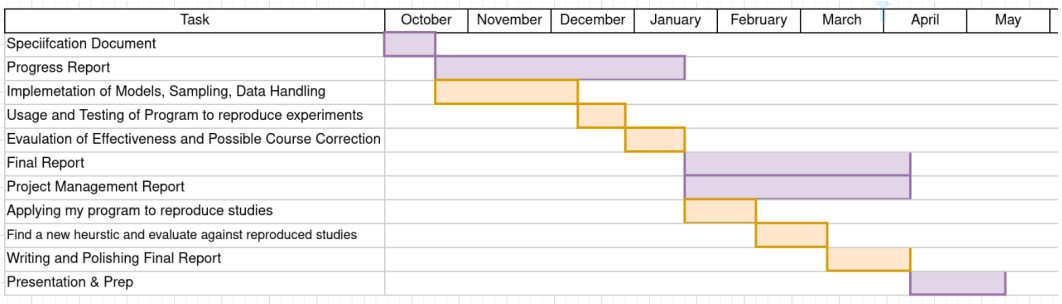


Figure 12: Gantt Chart of Expected Timeline

F Resources and Risks

F.1 Resources

1. Python and Standard Libraries - NetworkX, NumPy, Pandas, Matplotlib, Build tools. These are all commonly available on a variety of systems, are open source, and can be

built from source in case PyPI is unavailable.

2. Large Graph Datasets - Enron emails, airport traffic data, Gnutella data, Twitter tweet data and any other data sets as research progresses. The data is available from multiple sources on the internet, as well as being stored in the repository of my program under version control.

3. Batch Compute System - Department of Computer Science (DCS). The datasets I will be working with are large but not unmanageable on a typical desktop. The batch compute system will be used to learn and utilise good practice for scientific computing, as well as allowing for faster computation.

F.2 Risks

1. Failure to find a new heuristic or sampling method that is measurably better than those currently written about. In this case the project would still be a success as it would show that it allows for evaluation of heuristics effectively to be able to show that it is not statistically better.

2. Lost or damaged device containing code and documents. Using GitHub to be a remote mirror of local git repository and regularly committing code will prevent issues from lost work arising. DCS machines are more than adequate for processing needs.

G Legal, Social, Ethical and Professional Considerations

At this stage there are no legal, social, ethical or professional considerations to take into account. The project will not be using, processing or interacting with personal data or information.

References

[1] J. Archbold, S. Clohessy, D. Herath, N. Griffiths, and O. Oyebode. An agent-based model of the spread of behavioural risk-factors for cardiovascular disease in city-scale populations. *PLOS ONE*, 19(5):1–22, 05 2024.

[2] J. Archbold and N. Griffiths. Limiting concept spread in environments with interacting concepts. In *16th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2017)*, pages 1332–1340. IFAAMAS, 2017.

[3] W. Chen, Y. Wang, and S. Yang. Efficient influence maximization in social networks. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 199–208, 2009.

[4] D. Eppstein and J. Wang. A steady state model for graph power laws. *CoRR*, cs.DM/0204001, 2002.

[5] H. Franks, N. Griffiths, and S. S. Anand. Learning agent influence in mas with complex social networks. *Autonomous Agents and Multi-Agent Systems*, 28:836–866, 2014.

[6] M. Gjoka, M. Kurant, C. Butts, and A. Markopoulou. Walking in facebook: A case study of unbiased sampling of osns. pages 1 – 9, 04 2010.

[7] M. Gjoka, M. Kurant, C. T. Butts, and A. Markopoulou. Practical recommendations on crawling online social networks. *IEEE Journal on Selected Areas in Communications*, 29(9):1872–1892, 2011.

[8] S. Goyal and M. Kearns. Competitive contagion in networks, 2012.

[9] L. Jin, Y. Chen, P. Hui, C. Ding, T. Wang, A. Vasilakos, B. Deng, and X. Li. Albatross sampling: robust and effective hybrid vertex sampling for social graphs. 07 2011.

[10] D. Kempe, J. Kleinberg, and E. Tardos. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '03, page 137–146, New York, NY, USA, 2003. Association for Computing Machinery.

[11] D. Kempe, J. Kleinberg, and É. Tardos. Influential nodes in a diffusion model for social networks. In *international colloquium on automata, languages, and programming*, pages 1127–1138. Springer, 2005.

[12] J. Kleinberg. The small-world phenomenon: an algorithmic perspective. In *Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing*, STOC '00, page 163–170, New York, NY, USA, 2000. Association for Computing Machinery.

[13] M. N. Kuperman. Invited review: Epidemics on social networks. *Papers in physics*, 5(1):0–0, 2013.

[14] C.-H. Lee, X. Xu, and D. Y. Eun. Beyond random walk and metropolis-hastings samplers: Why you should not backtrack for unbiased graph sampling, 2012.

[15] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, and N. Glance. Cost-effective outbreak detection in networks. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '07, page 420–429, New York, NY, USA, 2007. Association for Computing Machinery.

[16] V. Sekara, A. Stopczynski, and S. Lehmann. Fundamental structures of dynamic social networks. *Proceedings of the national academy of sciences*, 113(36):9977–9982, 2016.

[17] T. Wang, Y. Chen, Z. Zhang, T. Xu, L. Jin, P. Hui, B. Deng, and X. Li. Understanding graph sampling algorithms for social network analysis. *2011 31st International Conference on Distributed Computing Systems Workshops*, pages 123–128, 2011.

[18] W. Yang, L. Brenner, and A. Giua. Influence maximization in independent cascade networks based on activation probability computation. *IEEE Access*, 7:13745–13757, 2019.